Capítulo 3

Fundamentos de redes neurais

Para Geron (2019), as redes neurais são um componente essencial ao aprendizado de máquina; pois, inspiradas no funcionamento do cérebro humano, têm a capacidade de modelar padrões complexos em dados. O autor, por meio de uma abordagem prática e sistemática, destaca as redes neurais como ferramentas poderosas para resolver problemas que vão desde a classificação de imagens até o processamento de linguagem natural, integrando-se a tecnologias modernas de inteligência artificial.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

Veremos os fundamentos das redes neurais, começando por sua estrutura básica, incluindo neurônios artificiais e camadas. Abordaremos os processos de treinamento, com foco em algoritmos de otimização, e discutiremos arquiteturas simples.

Além disso, analisaremos os desafios associados ao treinamento, como a inicialização de pesos e a qualidade dos dados. Por fim, destacaremos ferramentas práticas, que facilitam o desenvolvimento e a implementação de redes neurais, democratizando o acesso à construção de modelos avançados.

1 Estrutura das redes neurais

As redes neurais, conforme Geron (2019), são um pilar muito importante do aprendizado de máquina moderno, pois são inspiradas no funcionamento do cérebro humano. Elas são compostas por unidades computacionais chamadas neurônios artificiais, organizadas em camadas, que processam dados para aprender padrões complexos.

Vejamos agora os conceitos básicos de redes neurais, desde sua estrutura até os desafios de treinamento e as ferramentas práticas para implementação, com base em abordagens práticas e teóricas de diversos autores.

As redes neurais são sistemas computacionais inspirados no funcionamento do cérebro humano. Como já sabemos, são projetadas para processar informações por meio de unidades interconectadas chamadas neurônios artificiais, organizadas em camadas (Geron, 2019).

Essas camadas, que incluem as de entrada, as ocultas e as de saída, trabalham em conjunto para transformar dados brutos em previsões ou classificações, ajustando pesos durante o treinamento para capturar padrões complexos.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

A seguir, listaremos os componentes fundamentais das redes neurais, como o neurônio artificial, a organização em camadas e as funções de ativação, que permitem modelar relações não lineares em diversas aplicações, desde reconhecimento de imagens até previsão de séries temporais.

1.1 Neurônio artificial: conceitos básicos

Para Suave (2024), neurônio artificial é a unidade básica de uma rede neural que simula o comportamento de um neurônio biológico, recebendo entradas, processando essas entradas e gerando uma saída.

Matematicamente, para Mueller e Massaron (2018), um neurônio artificial opera da seguinte forma:

Dadas as entradas x1, x2, . . . , xn, cada uma associada a um peso w1, w2, . . . , wn, o neurônio calcula:

$$z = \sum_{n\,i=1} WiXi + b$$

Em que *b* é o termo de viés (bias). Essa soma é passada por uma função de ativação f(z), que introduz não linearidade, gerando a saída do neurônio:

$$a = f(z)$$

O viés permite que o modelo ajuste *a* mesmo quando as entradas são zero, aumentando a flexibilidade do aprendizado.

Segundo Suave (2024), esse componente, conhecido como bias, pode aumentar ou diminuir as chances de um neurônio ser ativado. Em resumo, os pesos (w) são responsáveis pela amplificação de sinais recebidos; já a função de ativação f(z) fica responsável por definir como um neurônio responde a esses sinais, tendo o controle da saída final de uma rede neural.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

As funções de ativação podem ser divididas em três partes: limiar, linear e Sigmoid.

- Liminar – em que o neurônio só será ativado se o sinal recebido for forte o suficiente.
- Linerar – em que é permitida uma resposta proporcional ao sinal.
- Sigmoids – responsável por modular a ativação de forma mais gradual e suave.

1.2 Camadas de entrada, ocultas e de saída

Segundo Mueller e Massaron (2018), uma rede neural é organizada em camadas, como podemos ver a seguir.

- Camada de entrada: recebe os dados brutos; por exemplo, pixels de uma imagem ou valores numéricos. Cada neurônio na camada de entrada representa uma característica do dado.
- Camadas ocultas: processam as entradas através de transformações aprendidas. O número de camadas ocultas e neurônios por camada define a complexidade do modelo.
- Camada de saída: produz o resultado final, como uma classificação; por exemplo, "gato" ou "cachorro" ou um valor numérico, como a previsão de preço.

IMPORTANTE

A comunicação entre camadas ocorre por meio de conexões ponderadas, em que os pesos são ajustados durante o treinamento para minimizar possíveis erros.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

1.3 Funções de ativação: Sigmoid, ReLU, Tanh, Softmax

Para Suave (2024), as funções de ativação introduzem não linearidade, permitindo que as redes neurais modelem problemas complexos. As mais comuns são:

- Sigmoid: mapeia valores para o intervalo (0, 1):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

O Sigmoid é útil em problemas de classificação binária, em transformações de valores ou em qualquer contexto em que valores precisem ser normalizados no intervalo de 0 até 1, ainda mantendo uma transformação proporcional ao valor original.

O -*z* representa a relevância de uma classe específica antes da conversão em probabilidade; como exemplo, uma rede neural que classifica imagens em "gato", "cachorro" e "pássaro". Esse pode ser o valor calculado para um desses elementos.

- ReLU (rectified linear unit): definida como:

$$f(z) = \max(0, z)$$

É computacionalmente eficiente e evita o problema do gradiente evanescente, sendo amplamente usada em redes profundas. O *z* representa a entrada da função, já o max(0,z) é a função de máximo que compara *z* com zero e retorna o maior valor entre eles; é a saída da função.

- Tanh: mapeia valores para o intervalo (−1, 1):

$$tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

É similar à Sigmoid, mas centrada em zero, o que pode facilitar o treinamento em alguns casos. É frequentemente usada nas camadas ocultas de redes neurais.

- Softmax – Usada em problemas de classificação multiclasse, converte um vetor de valores em probabilidades que somam 1:

$$\text{Softmax}(z_i) = \frac{e^{zi}}{\sum j^{ezj}}$$

Também frequentemente usada como uma última função de ativação. A escolha da função de ativação depende do problema e da arquitetura da rede; em que $\sum j^{ezj}$ é a soma de todas as exponenciais dos valores e *ezi* é a função exponencial aplicada.

2 Treinamento de redes neurais

Para Malik (2021), o treinamento de redes neurais é o processo central para que modelos aprendam a realizar tarefas específicas, ajustando os pesos das conexões entre neurônios para minimizar os erros entre as previsões e os valores reais.

2.3.1 Algoritmos de otimização: SGD, Adam, RMSProp

Como vimos em Malik (2021), o treinamento de redes neurais consiste em ajustar os pesos para minimizar uma função de perda, que mede o erro entre as previsões do modelo e os valores reais. Os algoritmos de otimização mais comuns incluem:

- SGD (stochastic gradient descent): atualiza os pesos com base no gradiente da função de perda para um subconjunto aleatório dos dados, também conhecido como minibatch:

$$w \leftarrow w - \eta \nabla L(w)$$

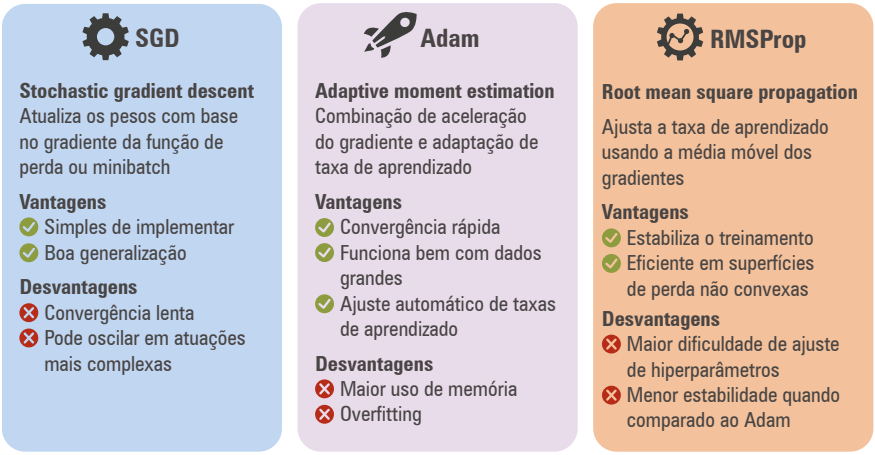
Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

Em que η é a taxa de aprendizado e ∇L(w) é o gradiente da perda. Simples, mas pode convergir lentamente.

- Adam (adaptive moment estimation): combina os benefícios do momento (aceleração do gradiente) e o RMSProp (adaptação da taxa de aprendizado por neurônio), amplamente utilizado devido à sua robustez.
- RMSProp: ajusta a taxa de aprendizado com base em uma média móvel dos quadrados dos gradientes, ajudando a estabilizar o treinamento em superfícies de perda não convexas.

Podemos ver na sequência da figura 1 uma comparação entre os algoritmos, mostrando algumas das vantagens e desvantagens no seu uso, além de algumas características que devemos levar em consideração para fazer a melhor escolha.

Figura 1 – Comparação de algoritmos de otimização



Segundo Suave (2024), a escolha do algoritmo de otimização impacta diretamente a velocidade e a qualidade da convergência.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

3 Arquitetura de redes simples

Segundo Geron (2019), as arquiteturas de redes neurais simples, como as redes feedforward e os perceptrons de múltiplas camadas, formam a base para a compreensão de modelos mais complexos em aprendizado de máquina, sendo amplamente utilizadas em tarefas como classificação e regressão

3.1 Redes feedforward: características e aplicações

Para Mueller e Massaron (2018), redes neurais feedforward são as arquiteturas do tipo mais simples, em que a informação flui em uma direção, da entrada para a saída. Elas são compostas por camadas densas, também chamadas de fully connected, e são adequadas para tarefas como:

- classificação de imagens simples;
- previsão de séries temporais;
- regressão linear e não linear.

Apesar de sua simplicidade, redes feedforward podem ser limitadas em tarefas que requerem memória ou processamento de dados sequenciais.

3.2 Redes de múltiplas camadas (MLPs)

As MLPs (multilayer perceptrons) são redes feedforward com uma ou mais camadas ocultas. Cada neurônio em uma camada está conectado a todos os neurônios da camada seguinte, formando uma estrutura mais densa.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

As MLPs são versáteis e podem modelar funções complexas; em contrapartida, elas requerem mais poder computacional, necessário à medida que o número de camadas aumenta. Ela foi desenvolvida com o objetivo de solucionar problemas complexos, até então não resolvidos pelo modelo de neurônio básico, como a falta de linearidade.

4 Desafios no treinamento de redes

Para Geron (2019), o treinamento de redes neurais, embora bastante eficaz, enfrenta desafios significativos que podem impactar a eficiência e a precisão dos modelos, como a inicialização inadequada de pesos, a dependência de grandes volumes de dados e a qualidade dos dados utilizados.

4.2.1 Inicialização de pesos

Segundo Malik (2021), a inicialização dos pesos é crucial para o treinamento. Pesos iniciais muito grandes ou muito pequenos podem levar a problemas como:

- Gradiente evanescente: quando os gradientes se tornam muito pequenos, dificultando o aprendizado.
- Gradiente explosivo: quando os gradientes crescem exponencialmente, causando instabilidade.

Técnicas como inicialização com Sigmoid, Tanh ou ReLU ajustam os pesos com base na dimensionalidade das camadas, promovendo convergência estável.

4.1 Necessidade de grande volumes de dados

Redes neurais, especialmente as profundas, requerem grandes quantidades de dados para generalizar bem. Com poucos dados, o modelo pode

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

sofrer de sobreajuste, também conhecido como overfitting, memorizando os dados de treinamento em vez de aprender padrões generalizáveis.

Técnicas como as de aumento de dados podem ser utilizadas. São usualmente chamadas de data augmentation ou regularização, e ajudam a mitigar esse problema.

4.2 Impacto da qualidade dos dados na precisão do modelo

Malik (2021) explica que a qualidade dos dados é tão importante quanto a quantidade. Dados ruidosos, incompletos ou enviesados podem levar a modelos imprecisos ou tendenciosos. Estratégias para melhorar a qualidade incluem:

- Pré-processamento, com a normalização e limpeza de outliers.
- Balanceamento de classes em problemas de classificação.
- Validação cruzada para avaliar a robustez do modelo.

5 Ferramentas e frameworks para redes neurais

As ferramentas e frameworks para redes neurais, como Keras e TensorFlow, desempenham uma tarefa muito importante para tornar mais popular o desenvolvimento de modelos de aprendizado de máquina. Segundo Geron (2019), permitem que tanto desenvolvedores iniciantes quanto especialistas construam e treinem redes neurais de forma eficiente.

Essas bibliotecas oferecem interfaces intuitivas e recursos avançados para criar arquiteturas, otimizar parâmetros e implementar soluções

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

em larga escala, sendo usualmente adotadas em aplicações acadêmicas e industriais.

Veremos um pouco sobre as funcionalidades de Keras, ideal para prototipagem rápida, e TensorFlow, voltado para implementações complexas, destacando como essas ferramentas simplificam o processo de construção de modelos robustos.

5.1 Uso de Keras para criação de redes simples

Segundo Geron (2019), o Keras é uma biblioteca de alto nível para construção de redes neurais. Foi originalmente desenvolvida por François Chollet em 2015 como um projeto de código aberto, com o objetivo de simplificar o desenvolvimento de modelos de aprendizado profundo.

Inicialmente projetado para funcionar com diferentes backends, como Theano e TensorFlow, o Keras foi integrado ao TensorFlow a partir da versão 2.0, tornando-se sua interface oficial de alto nível, o que aumentou sua popularidade devido à facilidade de uso e flexibilidade.

Ele permite criar redes neurais com poucas linhas de código, utilizando Python, e é ideal para desenvolvedores iniciantes e profissionais que buscam prototipagem rápida. Um exemplo prático de criação de uma rede neural feedforward (MLP) em Keras para um problema de classificação multiclasse é:

```
‘from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# Criando o modelo
model = Sequential([
   Dense(64, activation=’relu’, input_shape=(100,),
name=’camada_entrada’),
   Dense(32, activation=’relu’, name=’camada_oculta’),
```


Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

```
    Dense(10, activation='softmax', name='camada_saida')
    ])
    # Compilando o modelo
    model.compile(optimizer='adam',
       loss='categorical_crossentropy',
       metrics=['accuracy'])
    # Resumo da arquitetura do modelo
    model.summary()'
```

No exemplo que usamos, a função *Sequential* cria uma rede neural sequencial, em que as camadas são adicionadas em ordem. Já a camada *Dense* define camadas totalmente conectadas, com a função de ativação ReLU nas camadas de entrada e ocultas, e Softmax na camada de saída, para gerar probabilidades para 10 classes.

O modelo é compilado com o otimizador Adam, que tem a função de perda *categorical_crossentropy*. Aqui, é adequada para classificação multiclasse, além de ser a métrica de acurácia para avaliação.

Além disso, Keras permite personalizar arquiteturas com camadas como *Dropout* para regularização ou *BatchNormalization* para estabilizar o treinamento, além de oferecer APIs (application programming interface) para pré-processamento de dados, como *keras.preprocessing*, que facilita o carregamento e manipulação de conjuntos de dados.

PARA PENSAR

Com base no que você já viu sobre o Keras, você consegue imaginar mais possibilidades de uso dessa poderosa biblioteca? Você acredita que ele pode ser eficiente no desenvolvimento de redes neurais, na classificação binária, na classificação de multiclasse ou em regressão?

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

Sua integração com Python e TensorFlow torna o Keras uma ferramenta muito útil para experimentação rápida e implementação de redes neurais em aplicações práticas. Como exemplos de uso, podemos citar o reconhecimento de imagens e processamento de texto.

5.2 TensorFlow: implementações avançadas

Segundo Suave (2024), o TensorFlow é um framework de código aberto para aprendizado de máquina. Desenvolvido pelo Google Brain Team e lançado em 2015, oferece uma plataforma bastante completa para construir e treinar modelos complexos em grande escala.

Ainda segundo o autor, o TensorFlow é uma das bibliotecas de código aberto mais influentes na área da inteligência artificial quando usada de forma particular no aprendizado de máquina e redes neurais profundas.

Projetado para suportar desde experimentos acadêmicos até aplicações em produção, o TensorFlow destaca-se por sua flexibilidade, suporte à computação distribuída e capacidade de operar em diversas plataformas, como CPUs (central processing unit), GPUs (graphics processing unit) e TPUs (tensor processing unit).

Diferentemente do Keras, que prioriza a simplicidade, o TensorFlow permite implementações avançadas, como a criação de camadas personalizadas, funções de perda customizadas e uma otimização adaptativa. É amplamente utilizado em aplicações que exigem alto desempenho, como processamento de linguagem natural e visão computacional.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

NA PRÁTICA

Exemplo prático de implementação avançada: criação de uma camada personalizada para normalização de dados em uma rede neural, como no código a seguir:

```
‘import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Layer
import numpy as np
# Definindo uma camada personalizada
class CustomNormalizationLayer(Layer):
   def __init__(self, **kwargs):
      super(CustomNormalizationLayer,
self).__init__(**kwargs)
   def call(self, inputs):
      # Normalização personalizada: subtrai a
média e divide pelo desvio padrão
      mean = tf.reduce_mean(inputs, axis=0)
      std = tf.math.reduce_std(inputs, axis=0)
      return (inputs - mean) / (std + 1e-6)  #
Evita divisão por zero
# Criando dados sintéticos
X = np.random.randn(1000, 20)  # 1000 amostras, 20
características
y = np.random.randint(0, 2, (1000, 1))  # Rótulos
binários
# Criando o modelo com a camada personalizada
model = Sequential([
   CustomNormalizationLayer(input_shape=(20,),
name=’normalizacao_personalizada’),
   Dense(64, activation=’relu’,
name=’camada_oculta’),
    Dense(1, activation=’sigmoid’,
name=’camada_saida’)
])
# Compilando o modelo
model.compile(optimizer=tf.keras.optimizers.
Adam(learning_rate=0.01),
        loss=’binary_crossentropy’,
        metrics=[‘accuracy’])
```


Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

```
# Resumo do modelo
model.summary()
# Treinando o modelo
model.fit(X, y, epochs=5, batch_size=32,
verbose=1)'
```

No nosso exemplo, a camada personalizada *CustomNormalizationLayer* normaliza os dados de entrada subtraindo a média e dividindo pelo desvio-padrão, um processo útil para estabilizar o treinamento.

O TensorFlow permite integrar essa camada em um modelo *Sequential*, combinando-a com camadas densas padrão. Além disso, o *framework* suporta funcionalidades avançadas, como a criação de modelos com arquiteturas não sequenciais usando a API funcional ou subclasses de *tf.keras.Model*, além de otimizações para implantação em dispositivos móveis e servidores por meio de ferramentas como TensorFlow Lite e TensorFlow Serving. Essas características fazem com que o TensorFlow seja ideal para projetos que exigem personalização e escalabilidade.

Considerações finais

Neste capítulo, oferecemos um resumo sobre os fundamentos das redes neurais, desde a estrutura básica dos neurônios artificiais até os desafios de treinamento e as ferramentas práticas que podem ser utilizadas.

Vimos um pouco sobre as principais características de utilização do TensorFlow e do Keras e contamos um pouco sobre sua história e seus objetivos de uso. Por fim, pudemos ver alguns exemplos de código para sua aplicação.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.

Referências

GERON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**: Concepts, Tools, and Techniques to Build Intelligent Systems. [*S. l.*]: O'Reilly Media, 2019.

MALIK, H. *et al*. **AI and Machine Learning Paradigms for Health Monitoring System**. Varsóvia: Springer, 2021.

MUELLER, J. P.; MASSARON, L. **Artificial Intelligence For Dummies**. Hoboken: For Dummies, 2018.

SUAVE, A. A. **Inteligência artificial**. São Paulo: Editora Freitas Bastos, 2024.

Material para uso exclusivo de aluno matriculado em curso de Educação a Distância da Rede Senac EAD, da disciplina correspondente. Proibida a reprodução e o compartilhamento digital, sob as penas da Lei. © Editora Senac São Paulo.